People's Democratic Republic of Algeria
الجمهورية الجزائرية الديمقراطية الشعبية
Ministry of Higher Education and Scientific Research
وزارة التعليم العالي و البحث العلمي

University of Batna 2
Faculty of Mathematics and Computer Science
Department of Computer Science

Thesis

To obtain the degree of Master in Computer Science

Specialty: Artificial Intelligence and Multimedia

Sign Language Recognition: AI Gives Voice to the Mute

Presented by

Yasmine Mahri
Asma Rahmani

Under the supervision of

Mohammed A. Merzoug

Academic Year: 2023-2024

Abstract

This thesis explores the development and evaluation of a sign language recognition system tailored for both Arabic and English using cutting-edge machine learning and deep learning technologies. The research integrates a unique dataset collected via a Raspberry Pi camera, emphasizing system performance optimized for real-time applications. The findings enhance the understanding of gesture-based communication and contribute to the improvement of automated accessibility tools.

Keywords: sign language recognition; real-time accurate recognition; machine learning; deep learning; arabic sign language; american sign language; accessibility technologies.

Contents

Abstract . . . . . **I**

List of Figures . . . . . **V**

List of Tables . . . . . **VI**

List of Acronyms . . . . . **VII**

1 Introduction **1**
1.1 Research motivation 1
1.2 Methodology and contribution 2
1.3 Thesis structure 4

2 Background: Gesture and Sign Language Recognition **5**
2.1 Introduction 5
2.2 Sign Language Definition 5
2.3 Considered Sign Languages 6
2.3.1 Arabic Sign Language 6
2.3.2 American Sign Language 7
2.4 Recognition Systems and Approaches 8
2.4.1 Vision-based Systems 8
2.4.2 Machine Learning vs. Deep Learning 9
2.4.3 Hybrid Models: ML and DL Integration 10
2.4.4 Transfer Learning and Ensemble Methods 10
2.4.5 Advanced Techniques 11
2.5 Application Examples of Sign Language Recognition Technology 11
2.6 Conclusion 12

3 Datasets: Collection and Preparation **13**
3.1 Introduction 13
3.2 What is a Dataset? 13
3.3 Balanced vs. Imbalanced Dataset 14
3.4 Imbalance Ratio 14
3.5 Used Datasets 15
3.5.1 Kaggle ASL and ArSL Datasets 15
3.5.2 Custom ASL and ArSL Datasets 15
3.6 Dataset Comparison: Kaggle vs. Custom 16
3.7 Data Preparation and Preprocessing 16

3.8 Conclusion . . . 18

4 AI Models: Training and Evaluation . . . 19
4.1 Introduction . . . 19
4.2 Used AI Models: CNNs and ML . . . 19
4.2.1 VGG16 . . . 20
4.2.2 MobileNetV2 . . . 20
4.2.3 MobileNetV3 . . . 20
4.2.4 DenseNet . . . 20
4.2.5 EfficientNet . . . 20
4.2.6 Random Forest Classifier . . . 21
4.3 Training and Testing CNN Models on Kaggle ASL Dataset . . . 21
4.3.1 Model Training . . . 21
4.3.2 Model Evaluation . . . 22
4.4 Performance of MobileNetV2 on Kaggle and Custom Datasets . . . 23
4.5 Performance of Random Forest Classifier . . . 25
4.5.1 Model Training . . . 25
4.5.2 Real-Time Evaluation . . . 26
4.6 Hybrid Model: MobileNetV2 and Random Forest Classifier . . . 26
4.7 Final Model Choice . . . 27
4.8 Conclusion . . . 28

5 System Design and Implementation . . . 30
5.1 Introduction . . . 30
5.2 System Requirements . . . 30
5.2.1 Functional Requirements . . . 30
5.2.2 Non-Functional Requirements . . . 31
5.3 System Architecture . . . 32
5.3.1 High-Level Design . . . 32
5.3.2 Low-Level Design . . . 33
5.4 Hardware and Software Configuration . . . 34
5.4.1 Training Hardware . . . 34
5.4.2 Deployment Hardware . . . 34
5.4.3 Software Configuration . . . 35
5.4.4 software and harware setup . . . 35
5.5 Implementation Details . . . 35
5.5.1 Interface Development . . . 36
5.5.2 Deployment on Raspberry Pi . . . 36
5.6 Conclusion . . . 36

6 Results, Discussion, and Conclusion . . . 38
6.1 Introduction . . . 38
6.2 Evaluation Metrics . . . 38
6.3 Comprehensive Performance Evaluation . . . 39
6.3.1 Model Accuracy and Loss Metrics . . . 39
6.3.2 Confusion Matrices and Graphical Analysis . . . 39
6.4 Real-Time System Performance . . . 40

6.4.1 Interface Functionality and Real-Time Testing 40
6.4.2 Interface Overview for Hand Gesture Recognition System 41
6.5 Future Work 42
6.5.1 Potential Enhancements 42
6.5.2 Future Research Directions 43
6.6 Conclusion 43

7 Conclusions and Perspectives 45

List of Figures

2.1 Arabic Sign Language (ArSL) Illustration. 7
2.2 American Sign Language (ASL) Illustration. 8

4.1 Confusion matrix for MobileNetV2 on the Kaggle dataset. 24

5.1 High-Level Design diagram 32

6.1 Confusion Matrix for the Random Forest Classifier on the English custom dataset 40

List of Tables

4.1 Performance comparison of CNN Models on Kaggle's ASL Dataset: VGG16, MobileNetV2, MobileNetV3, DenseNet, EfficientNet. 23

5.1 Summary of Software and Hardware Configuration 35

List of Acronyms

AI	*Artificial Intelligence*
ArSL	*Arabic Sign Language*
ASL	*American Sign Language*
BSL	*British Sign Language*
CNN	*Convolutional Neural Network*
DL	*Deep Learning*
ML	*Machine Learning*
RGB	*Red, Green, Blue color model*
RPi	*Raspberry Pi*

Chapter 1

Introduction

In recent years, the rapid advancement of technology has significantly reshaped various aspects of daily life, including how we communicate. Among the numerous fields of research, gesture recognition technology stands out due to its profound potential to transform non-verbal communication. This technology, especially pivotal for individuals unable to use spoken language, offers a new avenue for interaction by converting hand gestures into text and speech. By enabling mute individuals to communicate more freely, gesture recognition technology fosters inclusivity and helps bridge communication gaps. This introductory chapter sets the stage for discussing the intricate challenges, methodologies, and potential impacts of the proposed system, providing a foundation for the detailed exploration that follows in subsequent chapters.

1.1 Research motivation

Communication is a fundamental human need, vital for personal relationships, education, and professional success. However, mute individuals face unique challenges in expressing themselves through traditional verbal means. While alternative conventional communication methods like sign language and text-based systems exist, they often require specific skills and are not universally understood. This creates barriers that can isolate mute individuals from engaging fully in societal interactions.

Gesture recognition technology has significantly transformed how people interact with devices and one another, proving particularly beneficial for those who rely on non-verbal communication. This technology employs sensors and advanced algorithms to translate human gestures into commands, enabling the control of devices or facilitating communication without the need for touch or voice. Examples such as the *Kinect* system, originally designed for gaming, have been adapted to assist in rehabilitation and physical therapy, providing mute individuals with a way to interact with computers through body movements. Additionally, specialized wearable devices like *smart gloves* convert sign language gestures into text or spoken commands, bridging communication gaps for those who use sign language.

Despite the progress made in gesture recognition technology, several challenges remain. Current systems often face issues such as varying accuracy, sensitivity to environmental factors, the need for user-specific calibration, and high computational demands. Moreover,

many systems come with a predefined set of gestures, limiting the scope of communication for users who may need more nuanced and complex interactions.

- *Accuracy and sensitivity:* one of the primary challenges with current gesture recognition systems is their variability in accuracy. These systems often struggle to consistently recognize gestures with high precision, especially in complex or dynamic environments. Factors like lighting conditions, background noise, and the user's distance from the sensor can significantly impact performance.
- *User-specific variations:* gesture recognition systems typically require calibration to individual users' movements, which can be a limitation in environments where multiple users interact with the system. People naturally have different gesture styles, speeds, and amplitudes, making universal recognition challenging without extensive customization.
- *Limited gesture libraries:* many systems are programmed with a limited set of predefined gestures, which can restrict the user's ability to communicate fully. This is particularly limiting in the context of sign language, where the diversity of gestures and their meanings is vast and nuanced.
- *Computational requirements:* effective gesture recognition requires significant computational power and sophisticated algorithms, which can be resource-intensive. This makes it difficult to implement these systems on low-power devices or in situations where processing resources are limited.
- *Accessibility and inclusiveness:* despite the potential of gesture recognition technology to aid communication for mute individuals, there is still a gap in ensuring that these systems are universally accessible. Many current systems are not designed with inclusivity in mind, often overlooking the needs of those who could benefit most from this technology.

The limitations of these existing methods underscore the urgency for innovative solutions. By addressing these challenges, future developments in gesture recognition technology can offer more reliable, inclusive, and user-friendly solutions.

1.2 Methodology and contribution

This research focuses on developing a gesture-to-text-to-speech translation system tailored to assist mute individuals, enhancing their ability to communicate more seamlessly and effectively. Several critical questions must be tackled to guide the development of such an innovative translation system.

- How can a gesture-to-text-to-speech translation system be designed and implemented to facilitate communication for mute individuals?
- What hardware and software components are necessary to develop an effective gesture recognition system capable of accurately translating gestures into text?

- How can datasets and machine learning techniques, specifically deep learning algorithms, be utilized to enhance the accuracy and efficiency of gesture recognition?

The adopted methodology involves a systematic approach to developing a real-time gesture-to-text-to-speech translation system, with a focus on recognizing gestures corresponding to both Arabic and English letters.

- Design and develop a robust gesture recognition system that accurately interprets and translates gestures into text and then speech in real time.
- The real-time system must be designed to be deployed on Raspberry Pis, therefore, it must be lightweight.
- Optimize system response time to minimize communication delays and enhance conversational flow without losing accuracy and precision.
- Integrate the gesture recognition system with a speech synthesis module for real-time generation of spoken language from the translated text.
- Expand the system's capabilities to recognize and interpret a diverse array of hand gestures, each corresponding to specific letters.

The project begins with the collection of a comprehensive dataset of hand gestures that represent the alphabet in the considered languages, Arabic and English. This dataset serves as the foundation for training both machine learning and deep learning models, ensuring the system can accurately recognize and interpret these specific gestures. The development process includes several stages:

- *Data collection and preparation:* gathering extensive gesture data for both Arabic and English alphabets.
- *Model training:* utilizing machine learning and deep learning techniques to develop robust models capable of recognizing gestures.
- *Testing and refinement:* conducting rigorous tests to refine the accuracy of gesture recognition.

The significance of this study lies not only in its technical achievements but also in its social implications. The key contributions of this work can be summarized in the following points:

- *Advanced gesture recognition for multiple languages:* by focusing on both Arabic and English, the system caters to a broader demographic, enhancing its utility and applicability.
- *Real-time processing:* the system is designed to interpret gestures into text and convert them to speech instantaneously, facilitating seamless communication.
- *Integration of machine learning and deep learning:* this dual approach not only heightens the accuracy of gesture recognition but also combines the strengths of both methods to optimize performance, ensuring reliable and effective translation of gestures into text and speech.

1.3 Thesis structure

This manuscript is organized into seven chapters, each addressing a specific aspect of the research:

- **Chapter 1 - Introduction:** This chapter provides an overview of the context, motivation, methodology, contribution, and structure of the thesis.
- **Chapter 2 - Background on Gesture and Sign Language Recognition:** This chapter explores the theoretical background and existing technologies in gesture recognition.
- **Chapter 3 - Dataset Collection and Preparation:** This chapter details the collection and preparation of the datasets used for training the models.
- **Chapter 4 - AI Models and Methodology:** This chapter discusses the machine learning and deep learning models used in system development.
- **Chapter 5 - System Design and Implementation:** This chapter describes the design and implementation of the gesture-to-text-to-speech system.
- **Chapter 6 - Results and Discussion:** This chapter presents and discusses the results and findings of the study.
- **Chapter 7 - Conclusion and Perspectives:** This last chapter concludes the manuscript by summarizing the key findings and contributions of the research. It also provides an overview of the limitations encountered during the study and offers recommendations for future research and development, highlighting the importance of continuous research in the field.

By structuring the manuscript in this way, we aim to provide a coherent narrative that guides the reader through the research process and findings, ultimately demonstrating the impact and significance of the developed gesture-to-text-to-speech translation system.

Chapter 2

Background: Gesture and Sign Language Recognition

2.1 Introduction

In this chapter, we explore the vital role of sign language recognition technology in aiding non-verbal communication. We begin by examining the characteristics of sign language, focusing particularly on its variations in the Arabic-speaking world and in American English contexts. Understanding these variations is crucial for developing technologies that can accurately interpret these languages. Next, we delve into the technologies that make sign language recognition possible. We discuss traditional and modern approaches, including machine learning and deep learning, and how they are applied to enhance the accuracy of recognizing sign language gestures. We also look at innovative methods like hybrid models, transfer learning, and ensemble methods, which combine multiple technological approaches to improve recognition systems. Additionally, we compare different camera technologies (RGB, depth, and infrared) that are used to capture sign language gestures. Each type of camera offers unique benefits and challenges in accurately interpreting gestures, and we assess their suitability for various environments and applications.

By examining these topics, this chapter aims to provide a comprehensive understanding of how sign language recognition technology works and its applications in real-world settings, making communication more accessible for mute individuals.

2.2 Sign Language Definition

Sign language is a powerful visual communication method that utilizes hand gestures along with facial expressions and body movements [15]. It is especially crucial for individuals who are mute and rely on non-verbal forms of communication to interact with the world around them. While the specifics of sign language can vary, the core idea remains the same: it is a system of manual signs used to express distinct meanings effectively [15].

Sign language is more than just a communication tool for individuals who are mute; it serves as a crucial bridge connecting them to the world [15]. While it opens many

doors, its necessity also underscores the significant barriers faced by the mute community. Sign language proves invaluable across various settings, alongside the challenges it helps mitigate, such as education, the workplace, healthcare, public services, entertainment, and media.

2.3 Considered Sign Languages

One important aspect to note is that sign language is not a one-size-fits-all solution [2]. Different communities have their own versions of sign language, each with its own set of signs and rules tailored to their cultural and regional context. For instance, American Sign Language (ASL) is quite different from British Sign Language (BSL), even though both countries primarily use English for spoken communication. This variety means that any effective recognition system needs to be adaptable (it should ideally adjust to different sign language dialects).

This project focuses on creating a system that not only recognizes the gestures of mute individuals but also adapts to various sign language systems, like Arabic Sign Language (ArSL), aiming to be as inclusive and useful as possible. Specifically, in this thesis, we consider two sign languages: Arabic Sign Language and American Sign Language.

2.3.1 Arabic Sign Language

Arabic Sign Language (ArSL) is not just one language; it is a collection of dialects spread across the Arabic-speaking world [20]. Each region has crafted its own set of signs and conventions, creating rich variations. Currently, there are movements toward standardizing ArSL to improve communication and accessibility. However, recognizing Arabic sign language presents unique challenges due to the diversity of its dialects and the intricate details in hand shapes and movements, which are critical to its structure. Figure 2.1 provides an illustration of Arabic sign language.

Figure 2.1: Arabic Sign Language (ArSL) Illustration.

2.3.2 American Sign Language

American Sign Language (ASL) is among the most widely recognized sign languages globally, primarily used in the USA and parts of Canada [24]. ASL is more than just a collection of signs; it has its own grammar, culture, and history, establishing it as a comprehensive language in its own right. For technological recognition systems, understanding ASL means decoding a complex array of gestures that encompass not only hand movements but also facial expressions and body postures. These elements are crucial as they contribute to grammatical and emotional nuances within the language.

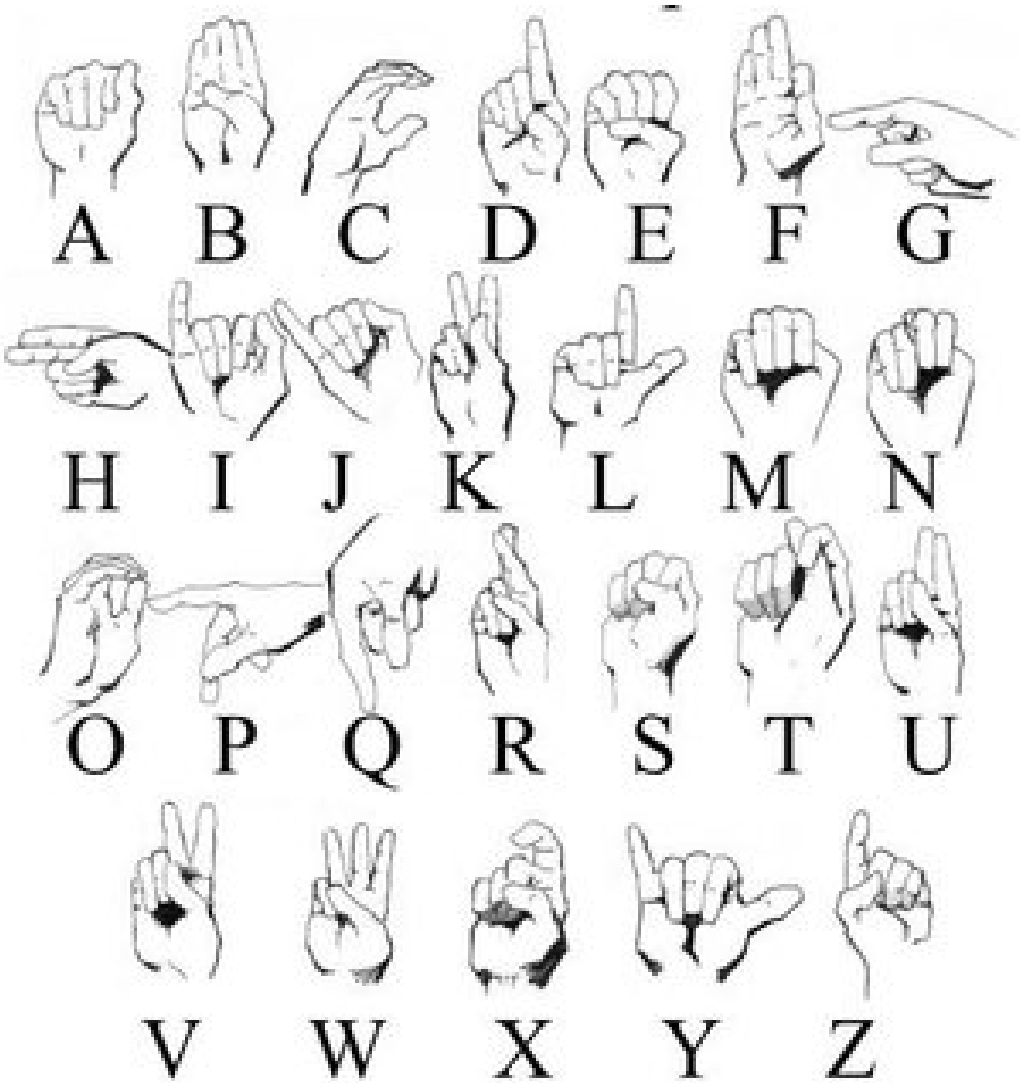

Figure 2.2: American Sign Language (ASL) Illustration.

2.4 Recognition Systems and Approaches

2.4.1 Vision-based Systems

For effective sign language recognition, the choice of camera technology is crucial, as it directly impacts the accuracy and functionality of the recognition system. Here, we explore the types of cameras used, with a particular focus on the RGB camera due to its relevance to our project setup.

RGB Cameras

- *Usage:* RGB cameras capture images in the red, green, and blue color spectrum, providing detailed visual data crucial for processing visible light images. In this project, we utilized an RGB camera, specifically the Raspberry Pi camera, which is inherently suited for capturing high-resolution color images.

- *Advantages:* RGB cameras are widely available, cost-effective, and compatible with a variety of computing devices, making them ideal for applications needing broad accessibility. They provide detailed color information that is essential for distinguishing subtle differences in hand gestures.
- *Limitations:* the primary limitation is their inability to capture depth information, which can be crucial for understanding the spatial aspects of complex gestures.

Depth Cameras

- *Usage:* depth cameras, such as those using Time-of-Flight (ToF) or structured light technologies, provide depth information by capturing the distance of objects from the sensor, creating a 3D map of the scene.
- *Advantages:* they excel in scenarios where understanding the depth and volume of the gestures is necessary, adding a layer of spatial data that enhances recognition accuracy.
- *Limitations:* depth cameras are typically more expensive and demand more computational resources, making them less ideal for cost-sensitive or mobile deployments.

Infrared Cameras

- *Usage:* infrared cameras are used to enhance visibility and recognition in low-light conditions by capturing infrared light emitted or reflected by objects.
- *Advantages:* they perform well in varied lighting conditions, providing consistent image quality, which is crucial for continuous gesture recognition.
- *Limitations:* their effectiveness decreases in outdoor environments where sunlight introduces infrared noise, potentially degrading the recognition accuracy.

The choice of the RGB camera for this project was driven by the use of the Raspberry Pi camera module, which is an RGB type. This camera was selected for its ease of integration with the Raspberry Pi hardware, facilitating the development of a portable and efficient sign language recognition system.

By focusing on RGB cameras, particularly the Raspberry Pi camera, this project leverages the balance between cost, availability, and sufficient technological capability to recognize sign language gestures effectively. The use of RGB cameras aligns with the project's goals to develop an accessible and practical sign language recognition system.

2.4.2 Machine Learning vs. Deep Learning

Machine learning (ML) empowers computers to interpret data and make decisions [17]. In sign language recognition, Random Forest classifiers are particularly effective. These classifiers, which aggregate decisions from multiple decision trees, are well-suited for managing

the complexities and variability of sign language gestures. They require low computational power and have demonstrated strong performance in accurately recognizing diverse sign language gestures, making them a practical choice for systems with limited resources.

Deep learning, a subset of ML, uses neural networks with many layers (hence "deep") to process data [14]. This approach excels at handling large volumes of data and can learn features directly from the data without needing manual feature extraction [14]. For sign language recognition, deep learning can automatically detect intricate patterns in gestures, making it highly effective but also demanding in terms of computational resources [14].

Machine learning models are generally less computationally intensive. Deep learning models, while requiring significant computational resources, excel at handling large, complex datasets and can automatically detect features, leading to higher accuracy and better generalization. However, the choice between ML and DL depends on the specific requirements and constraints of the application. This approach excels at handling large volumes of data and can learn features directly from the data [7].

2.4.3 Hybrid Models: ML and DL Integration

By integrating traditional ML techniques with deep learning, the system can harness the strengths of both. For instance, initial gesture features might be refined using ML methods before being fed into a deep learning model for further classification. This integration can help in improving accuracy while managing the computational load more effectively.

Hybrid models that combine ML and DL offer a balanced approach, enhancing recognition accuracy. These models leverage the quick adaptability of ML for recognizing basic gesture patterns and the powerful feature-learning capabilities of DL for more complex recognition tasks. This synergy creates a robust system capable of interpreting a wide range of sign language gestures with high precision.

2.4.4 Transfer Learning and Ensemble Methods

Transfer learning is a technique where a model developed for one task is reused as the starting point for a model for a second task. For sign language recognition, this can mean using a pre-trained model, perhaps developed for image recognition, and fine-tuning it to recognize sign language gestures. This method significantly reduces the time and data needed to train effective models.

Ensemble methods, where multiple models are used together to make predictions, can also be applied to sign language recognition. By combining predictions from various models, ensemble methods can improve accuracy and reliability, compensating for any one model's weaknesses.

These advanced techniques are particularly useful in sign language recognition, where the diversity of gestures and subtle differences between similar signs can pose a challenge. Hybrid models, transfer learning, and ensemble methods all contribute to creating more adaptable and accurate recognition systems, making technology more accessible and effective for users in real-world scenarios.

2.4.5 Advanced Techniques

Advanced techniques for sign language recognition include semantic segmentation, temporal modeling, and the integration of both approaches.

- *Semantic segmentation*: plays a critical role in improving the accuracy of sign language recognition systems [19]. This technique involves dividing an image or video frame into segments that represent different categories, such as hands, fingers, and facial expressions, which are essential components in sign language. By accurately segmenting these parts, the system can focus on analyzing the specific features of each segment, leading to a more precise interpretation of gestures.

  For instance, in a sign language recognition system, semantic segmentation allows the algorithm to distinguish between the hand and the background, or between different parts of the hand. This detailed analysis helps in accurately capturing the nuances of each sign, especially those that rely heavily on the position and movement of fingers.

- *Temporal modeling:* focuses on recognizing objects and their positions in images [3]. Temporal modeling is needed to understand actions that happen over time, which is crucial for interpreting sign language. Sign language involves movements and transitions, not just static poses.

  Temporal modeling techniques like Hidden Markov Models (HMMs) or Long Short-Term Memory networks (LSTMs) are used to identify patterns in sequences. These models help the recognition system understand both individual signs and the sequence of gestures over time. This is important for recognizing phrases and sentences in sign language, where the context and transitions between signs can change the meanings.

- *Integration of semantic segmentation and temporal modeling*: integrating semantic segmentation with temporal modeling provides a comprehensive approach to sign language recognition. This integration enables the system to accurately capture and analyze the complex interplay between the spatial features of individual gestures and their temporal evolution. Such advanced techniques significantly enhance the system's ability to interpret a wide array of sign language nuances accurately, making it more reliable and effective for users in diverse communication scenarios [8].

2.5 Application Examples of Sign Language Recognition Technology

Sign language recognition technology has opened up a world of possibilities for enhancing communication, accessibility, and interaction for the mute community. Here are some key areas where this technology is making a significant impact:

- *Education:* this technology transforms classrooms by enabling real-time translation of sign language into text or speech, making lessons accessible and allowing mute students to participate actively in discussions.

- *Customer service:* in settings like banks and hospitals, sign language recognition systems facilitate interactions between staff and mute customers. These systems allow for autonomous transactions and service access, eliminating the need for human interpreters.
- *Telecommunications:*the technology empowers mute individuals to make video calls where their sign language gestures are instantly translated into text or speech, enabling seamless long-distance communication.
- *Public safety and emergency services:* in emergencies, sign language recognition systems ensure that mute individuals can quickly and clearly communicate with responders, enhancing their safety and ensuring timely assistance.
- *Entertainment and media:*Sign language recognition is used to automatically translate spoken dialogue into sign language in media, making content like TV shows and movies accessible to mute viewers, promoting inclusivity in entertainment.

2.6 Conclusion

In this chapter, we explored essential aspects of sign language recognition. We defined sign language and examined its variations, such as Arabic and American sign languages, highlighting the need for adaptable recognition systems. We reviewed several technological systems and approaches used in sign language recognition, including vision-based systems (RGB, depth, and infrared), machine learning, and deep learning, discussing their roles in enhancing gesture accuracy. We also looked at hybrid models, transfer learning, ensemble methods, and other advanced techniques that improve the system's effectiveness.

In the next chapter, we shift the focus to the practical applications of these technologies, specifically looking at the dataset aspect. In particular, we will delve into the creation and preparation of datasets essential for training accurate recognition systems. This discussion will include challenges in data collection, methods for dataset preparation, and comparisons between existing and custom datasets. This next chapter aims to provide a thorough understanding of the data behind sign language recognition systems.

Chapter 3

Datasets: Collection and Preparation

3.1 Introduction

In this chapter, we focus on the backbone of any sign language recognition system: the dataset. Selecting the right dataset is crucial because it directly influences the effectiveness and accuracy of the recognition technology. We explore various datasets used for sign language recognition, particularly examining how they have been prepared and processed to train robust models. We begin by discussing the concept of dataset imbalance, an often encountered challenge in machine learning where some classes are overrepresented compared to others. Understanding and addressing this imbalance is vital for training models that perform well across all signs, not just the most common ones.

Furthermore, we delve into the details of the datasets that were selected for this project, including those sourced from Kaggle for both American and Arabic sign languages, as well as custom datasets that were created to address specific challenges encountered with pre-existing data. We explore why these datasets were chosen, how they were collected, and the specific preprocessing techniques employed to make them suitable for training sophisticated deep learning models. Lastly, we provide a preview of the methods used to split these datasets into training, validation, and test sets, setting the stage for their use in training the models discussed in the next chapter.

3.2 What is a Dataset?

In machine learning, a dataset is a collection of data that the model learns from. Choosing the right dataset is crucial because the quality and type of data we use can significantly determine how well the model performs.

A dataset typically consists of numerous examples, each belonging to a certain category or class. For instance, in sign language recognition, each dataset might include various gestures that correspond to different alphabetical letters or words.

3.3 Balanced vs. Imbalanced Dataset

Dataset imbalance occurs when the number of examples in each class within a dataset is not evenly distributed [6]. This means some classes have a lot more samples than others. Imagine trying to teach someone new vocabulary words, but you only focus on some words much more than others. Naturally, they will become better at recognizing the words they hear more often. Similarly, an imbalanced dataset can make a machine learning model perform well on more frequent classes while struggling with less common ones [6].

The primary impact of an imbalanced dataset is on the model's ability to learn and perform evenly across all classes. This is particularly problematic in applications like sign language recognition, where missing out on accurately recognizing less frequent signs could significantly hinder effective communication. An imbalanced dataset can lead to biased models that are less reliable, which might end up prioritizing accuracy on dominant classes at the expense of overall performance.

A balanced dataset ensures that the model gets a fair representation of all classes, which is crucial for achieving high accuracy and robustness in real-world applications [4]. It helps the model learn to recognize and interpret less frequent signs as accurately as the more common ones, which is vital for building inclusive technologies that cater effectively to all users.

Choosing the right dataset involves considering the diversity and comprehensiveness of the data and ensuring it reflects real-world scenarios where the model will be deployed. This is why we often turn to creating custom datasets when available ones do not meet the necessary criteria, ensuring that the data used for training is as close as possible to what the model will encounter outside the training environment.

3.4 Imbalance Ratio

The imbalance ratio is a simple yet powerful metric that helps us understand the extent of imbalance within a dataset [12]. It is calculated by dividing the number of samples in the largest class (majority class) by the number of samples in the smallest class (minority class) [12]. For example, if the largest class in your dataset has 1,000 samples and the smallest has 100, the imbalance ratio would be 10. This means that the largest class appears ten times more frequently than the smallest class.

Calculating the imbalance ratio gives us a quick snapshot of the dataset structure and highlights potential challenges in model training [6]. A high imbalance ratio can lead to a model that is biased towards the majority class, as it has more examples to learn from, often at the expense of minority classes. This can result in poor model performance, especially when it encounters less-represented classes in real-world situations [9].

In practical terms, knowing the imbalance ratio allows data scientists and developers to take steps to correct this imbalance before training begins, and that is what we did in our case with the datasets that are not balanced. Techniques such as resampling the data, either by oversampling the minority classes or undersampling the majority classes, or applying different weights to classes during model training are common approaches to

mitigate the effects of a high imbalance ratio [6].

By addressing dataset imbalance effectively, we ensure that the model learns to recognize all classes with equal precision, enhancing its accuracy and fairness [6]. This is especially critical in applications like sign language recognition, where missing or misinterpreting signs could significantly impact the usability of the technology.

3.5 Used Datasets

3.5.1 Kaggle ASL and ArSL Datasets

In this project, we utilize two significant datasets from Kaggle [13], which serve as the foundational data for training our sign language recognition models. These datasets were chosen for their comprehensive collection of sign language gestures, representing both American and Arabic sign languages.

- *American sign language dataset* [16]: is particularly valuable because it includes a wide variety of ASL signs, making it one of the largest of its kind. It contains thousands of images that represent the different gestures used in ASL, each labeled with the corresponding letter or number. This extensive collection allows for robust training of models, especially convolutional neural networks (CNNs), which perform well with large datasets.
- *Arabic sign language dataset* [1]: similar to the ASL dataset, the Arabic Sign Language dataset features a vast array of hand gestures representing the Arabic alphabet captured in RGB format. The dataset is crucial for developing recognition systems that are specifically tailored to the nuances of Arabic sign language, accommodating the unique gestures and signs used in this language.

Both datasets are instrumental for our research as they provide a diverse range of examples, which is crucial for training effective sign language recognition systems. The availability of such comprehensive data allows us to implement and test various machine learning models to find the most effective approach for accurate and real-time sign language interpretation.

3.5.2 Custom ASL and ArSL Datasets

While existing datasets from platforms like Kaggle [13] provide a valuable foundation for training sign language recognition models, they come with certain limitations that can hinder their effectiveness in real-world applications [5]. These challenges include camera compatibility issues and variations in environmental conditions such as lighting and background, which are often not adequately represented in pre-existing datasets. These discrepancies can lead to problems like overfitting, where a model performs well on training data but fails to generalize to new, unseen data captured in different conditions. To address these issues and better prepare our models for practical deployment, we created two custom datasets.

- *Web camera dataset*: created using a standard web camera *for both languages*, this dataset aims to mirror the typical usage environments for sign language recognition software, such as homes or offices. This setup allows the models to be exposed to and trained under variable lighting conditions and diverse backgrounds, which are common in everyday scenarios [5].

- *Raspberry Pi camera dataset*: created using a Raspberry Pi camera *for both languages*, this dataset is tailored to optimize performance in cost-effective technology solutions, particularly for educational or community settings where budget constraints are significant. Training with these datasets ensures that the models are not only affordable but also effective in real-time applications on similar hardware setups, promoting a seamless transition from training to deployment [5].

The decision to develop these custom datasets was driven by the need to enhance the robustness and usability of our models across various real-world environments. By training with data that closely mimics the actual operational settings, including managing variations in camera specifications and environmental conditions, our models can achieve higher accuracy and better generalization [5]. This approach significantly mitigates the risk of overfitting and prepares the system for effective real-time performance, addressing the critical limitations observed in the pre-existing datasets.

3.6 Dataset Comparison: Kaggle vs. Custom

When developing a sign language recognition system, the choice of dataset can significantly impact the performance and effectiveness of the final model. Here is how our custom datasets stand out compared to the existing datasets sourced from Kaggle [13].

One of the primary advantages of our custom datasets is their balance. Unlike the existing dataset (Arabic Alphabets Sign Language), which often has a disproportionate number of samples for certain signs, our custom datasets were carefully curated to ensure a more uniform distribution of samples across all categories. This balance is crucial for preventing model bias towards more frequently represented signs, thereby enhancing the overall accuracy and fairness of the recognition system.

The custom datasets were specifically designed with the project's unique requirements in mind. For instance, since the real-time output will be deployed on a Raspberry Pi, the dataset created using the Raspberry Pi camera provides invaluable insights into how the model performs with the actual deployment hardware. This direct relevance ensures that the models are not only theoretically effective but also practically viable in the intended operational environment.

3.7 Data Preparation and Preprocessing

Preparing all the datasets for training involved a series of crucial preprocessing steps. Each of these steps helps transform raw data into a format that is optimal for training

machine learning and deep learning models [21]. Here is how our data was preprocessed.

- *Image resizing:* the first step was to standardize the size of images across all datasets. Resizing images ensures that all input data to the model is uniform, which is important for both training consistency and computational efficiency [21].
- *Image conversion to arrays:* after resizing, images were converted into arrays. This conversion facilitates the manipulation and processing of image data in numerical format, which is necessary for the subsequent steps and is a standard practice in machine learning workflows [21].
- *Data normalization:* normalization was applied to the image arrays to scale the pixel values to a range of 0 to 1. This process helps speed up convergence during training by ensuring that each input parameter (pixel, in this case) has a similar data distribution. This is particularly important for neural network efficiency [21].
- *Color channel adjustment:* for our datasets, we adjusted the color channels specifically from RGB (Red, Green, Blue) to BGR (Blue, Green, Red) format. This adjustment is necessary because different software and libraries expect image data to be in specific color formats. For instance, certain popular machine learning libraries default to the BGR format for historical reasons related to how image data is stored in memory. Ensuring that all images are in BGR format helps maintain consistency in how the model processes color data, avoiding any potential discrepancies that could adversely impact the model's ability to accurately recognize sign language gestures [21].
- *Label creation:* each image was labeled with the corresponding sign language gesture. Labeling is essential for supervised learning as it provides the ground truth that the model will learn to predict [21].
- *Dataset splitting:* the used datasets were split into three parts: 80% for training, 10% for validation, and 10% for testing. This split helps in training models on a large portion of data while also keeping separate sets for tuning and evaluating model performance, which is critical for an unbiased assessment [21].
- *Train set augmentation:* data augmentation techniques such as *random rotations*, *shifts*, and *flips* were applied, but only to the training set. Augmentation artificially expands the training dataset and helps the model generalize better by simulating a broader range of scenarios that it might encounter in real-world applications.
- *Dataset batching:* batching in machine learning involves grouping samples into manageable sets, and for this project, we used a batch size of 32. This size is optimal for balancing memory management and computational efficiency, which is crucial for training models on devices with limited resources like the Raspberry Pi. A batch size of 32 helps improve the training dynamics by allowing frequent parameter updates, which enhances the model's ability to learn and generalize from the training data. This setup ensures the system is both efficient and effective during real-time applications, making it well-suited for the real-world deployment of our sign language recognition system.performance [21].

3.8 Conclusion

In this chapter, we delved into the crucial aspects of datasets and data preparation that underpin the development of efficient, lightweight sign language recognition models. We began by addressing the issue of dataset imbalance and explaining its implications and how it affects model training. This discussion highlighted the importance of having a balanced dataset to ensure fair and accurate model performance across all sign classes. We then explored the diverse datasets used in this project, including those sourced from Kaggle and the custom datasets we created. The Kaggle datasets provided a solid foundation, while our custom datasets, developed using web cameras and Raspberry Pi cameras, were tailored to overcome specific challenges such as overfitting and to enhance the model's ability to perform in real-world settings. The process of creating these datasets involved meticulous data collection and preprocessing efforts, including resizing images, normalizing data, adjusting color channels, and more. These steps were critical to preparing the data adequately and ensuring that the models trained are robust and capable of handling diverse environmental variables.

In the next chapter, we shift our focus from data preparation to the heart of our project, model training. In particular, we will explore the various machine learning models employed, delve into their training processes, and discuss the techniques used to optimize their performance. This upcoming chapter will provide deeper insights into the technical workings of the models and how they were fine-tuned to achieve high accuracy and efficiency in sign language recognition.

Chapter 4

AI Models: Training and Evaluation

4.1 Introduction

In this chapter, we delve into the heart of our research on sign language recognition, exploring a range of modeling techniques that address the unique challenges presented by this form of communication. The primary goal is to assess and compare different models' effectiveness in accurately interpreting sign language through visual inputs.

We begin our exploration with convolutional neural networks (CNNs), a class of deep neural networks renowned for their prowess in image recognition and processing tasks. CNNs are particularly adept at picking out intricate patterns in visual imagery, making them a prime candidate for recognizing the subtle nuances of sign language gestures. However, while CNNs offer robust capabilities, they are not without limitations, especially when dealing with real-time data and diverse environmental conditions. To address these challenges, we also explore traditional machine learning methods, which can offer greater simplicity and efficiency in some scenarios. Specifically, we investigate the effectiveness of Random Forest classifiers, known for their high accuracy in classification tasks involving complex and varied data sets. Recognizing the potential strengths and weaknesses of both approaches, we also experiment with hybrid models that combine the feature extraction capabilities of CNNs with the classification power of machine learning techniques like Random Forest. This hybrid approach aims to leverage the best features of both realms to enhance the accuracy and real-time performance of sign language recognition systems.

Throughout this chapter, we detail each model's training process, evaluate their performance, and discuss the practical implications of our findings. By comparing these models side-by-side, we aim to determine the most effective approach for translating the silent gestures of sign language into a form that can be easily understood by computers, thereby bridging communication gaps and fostering inclusivity.

4.2 Used AI Models: CNNs and ML

In our exploration of efficient models for sign language recognition, we employed several advanced AI models, each with distinct features and specific applications. Below is a brief

overview of each model employed in our research.

4.2.1 VGG16

Developed by researchers at Oxford, VGG16 or very deep convolutional networks [22] is a deep convolutional neural network known for its simplicity and depth. It consists of 16 layers and is highly regarded for its ability to recognize features in images due to its deep architecture. VGG16 has been widely used in image recognition tasks and serves as a strong baseline for many visual recognition challenges.

4.2.2 MobileNetV2

MobileNetV2 [18] is designed to be both lightweight and efficient, making it suitable for applications that require operating on mobile devices or environments with limited computational resources. MobileNetV2 improves upon its predecessor by using inverted residuals and linear bottlenecks to enhance model efficiency without sacrificing accuracy, which is crucial for real-time sign language recognition.

4.2.3 MobileNetV3

An iteration on the MobileNet series, MobileNetV3 [10] applies a combination of hardware-aware network architecture search and traditional tuning techniques to optimize performance. It is even more efficient than MobileNetV2 [18] and is tailored for mobile and resource-constrained environments, making it an excellent candidate for applications needing high efficiency and real-time processing.

4.2.4 DenseNet

DenseNet (short for densely connected convolutional networks) [11] is unique for its architecture, where each layer is directly connected to every other layer in a feed-forward fashion. This model is particularly effective at reducing the vanishing gradient problem, strengthening feature propagation, and encouraging feature reuse, which can be beneficial for the detailed feature extraction required in sign language gestures.

4.2.5 EfficientNet

Known for scaling up CNNs in a more structured and effective way, EfficientNet [23] uses a compound coefficient to uniformly scale the depth, width, and resolution of the network. This model provides a state-of-the-art trade-off between accuracy and efficiency, making it suitable for scenarios where computational resources are a consideration but high accuracy is still required.

4.2.6 Random Forest Classifier

The Random Forest classifier [25] is a robust machine learning algorithm known for its simplicity and effectiveness in classification tasks. It operates by constructing multiple decision trees during the training process and outputting the class, which is the mode of the classes (classification) or mean prediction (regression) of the individual trees. Random Forests perform well on a wide range of data sets because they are capable of handling high-dimensional data and maintaining accuracy even when a large proportion of the data is missing.

Each model was carefully selected to address specific aspects of the sign language recognition challenge. CNNs, with their deep learning capabilities, are ideal for extracting detailed features from image data, while the Random Forest classifier offers fast, reliable classification across various conditions. By evaluating these models both individually and in combination, we aim to harness their strengths to create a versatile, accurate system capable of real-time sign language interpretation.

4.3 Training and Testing CNN Models on Kaggle ASL Dataset

CNNs were chosen for initial tests due to their proven strength in handling image recognition tasks, which is central to sign language interpretation. Each selected model has distinct features.

- *VGG16* [22]: is known for its deep network, which is capable of capturing complex patterns but can be computationally intensive.
- *MobileNetV2* [18] *and MobileNetV3* [10]: offer a balance between performance and computational efficiency, making them ideal for applications where speed and low resource use are critical.
- *DenseNet* [10]: provides excellent connectivity among layers, which promotes feature reuse and is beneficial for the nuanced features of sign language.
- *EfficientNet* [23]: scales uniformly across dimensions (depth, width, and resolution), providing high efficiency and strong accuracy at lower computational costs.

4.3.1 Model Training

The models were trained using the large and diverse American Sign Language dataset available on Kaggle [16]. This dataset is particularly well-suited for CNNs due to its extensive size and the variety of signs it includes, which allows the networks to learn a broad spectrum of features. The training process involved:

- *Data preprocessing:* all images in the dataset were resized to uniform dimensions of 224x224 pixels to ensure consistency in input size for all data. This resizing is

crucial for maintaining computational efficiency, particularly when processing data on a CPU. Additionally, images were normalized using TensorFlow's preprocess input function, which adjusts the color values to fit the MobileNetV2 model's expected input range. This normalization helps to standardize the value scales across the dataset, optimizing the model's ability to learn from the data effectively and efficiently.

- *Data augmentation:* data augmentation techniques such as *rotations*, *width and height shifts*, and *horizontal flipping* were employed to enhance the dataset's robustness and prevent overfitting. These techniques also help the model generalize better by simulating various real-world conditions.

- *Training settings:* Given the CPU's limitations, a batch size of 32 was selected to balance memory use and training speed efficiently. This size ensures that the training process runs smoothly without memory overflows. Additionally, the *Adam* optimizer was used for its adaptive learning rate, enhancing training effectiveness even with the CPU's slower processing capabilities. This approach optimizes both learning stability and speed, crucial for effective model training on limited hardware.

4.3.2 Model Evaluation

In this section, we evaluate the performance of the CNN models trained on the American Sign Language dataset from Kaggle [16]. The focus is primarily to demonstrate why MobileNetV2 [18] was chosen as the most efficient among the contender models, including VGG16 [22], MobileNetV3 [10], DenseNet [10], and EfficientNet [23].

- *MobileNetV2* exhibited outstanding performance with an accuracy of 99.79%, the highest among all tested models (Table 4.1). This high accuracy, coupled with its computational efficiency, made it the ideal choice for further development.

- *MobileNetV3* followed with a solid accuracy of 91.39%. Although it performed well, it did not match the efficiency and accuracy of MobileNetV2, especially when considering deployment in resource-constrained environments.

- *DenseNet* achieved a moderate accuracy of 76.5%. While respectable, it was not sufficient for the project's needs, where higher accuracy was crucial.

- *VGG16 and EfficientNet* had lower performances, with accuracies of 54.8% and 31.2%, respectively. These results indicated potential challenges in their architectures for handling the dataset's variability effectively.

Model	Accuracy
VGG16	54.8%
MobileNetV2	99.79%
MobileNetV3	91.39%
DenseNet	76.5%
EfficientNet	31.2%

Table 4.1: Performance comparison of CNN Models on Kaggle's ASL Dataset: VGG16, MobileNetV2, MobileNetV3, DenseNet, EfficientNet.

Detailed Analysis of MobileNetV2 Results

- *Accuracy metrics:* MobileNetV2 not only scored high on accuracy but also excelled in precision, recall, and F1-score, achieving perfect scores in controlled testing conditions.
- *Training and validation loss:* with a training loss of 0.0875, MobileNetV2 demonstrated effective learning capabilities, managing to capture the essential patterns in the data without overfitting.
- *Confusion matrix:* the confusion matrix for MobileNetV2 showed very high true positive rates for nearly all classes and minimal confusion between classes, underscoring its superior classification ability across diverse sign gestures.

Despite its high performance in test conditions, initial real-time testing of MobileNetV2 revealed some limitations, which prompted the development of a custom dataset more aligned with practical deployment scenarios.

The comprehensive performance of MobileNetV2, particularly its impressive accuracy and efficiency, made it the model of choice for advancing into further optimization and application on custom datasets designed to mimic real-world operational conditions.

4.4 Performance of MobileNetV2 on Kaggle and Custom Datasets

The performance of MobileNetV2 was rigorously evaluated on two different datasets: the American Sign Language dataset from Kaggle and a custom dataset that collects with the Raspberry Pi camera designed to address specific challenges encountered during initial real-time testing. This comparison is crucial to understanding the model's adaptability and effectiveness under varying conditions.

Performance on Kaggle ASL Dataset

On the Kaggle dataset, MobileNetV2 achieved an impressive accuracy of 99.79%. While this demonstrates the model's capability in a controlled, large-scale dataset

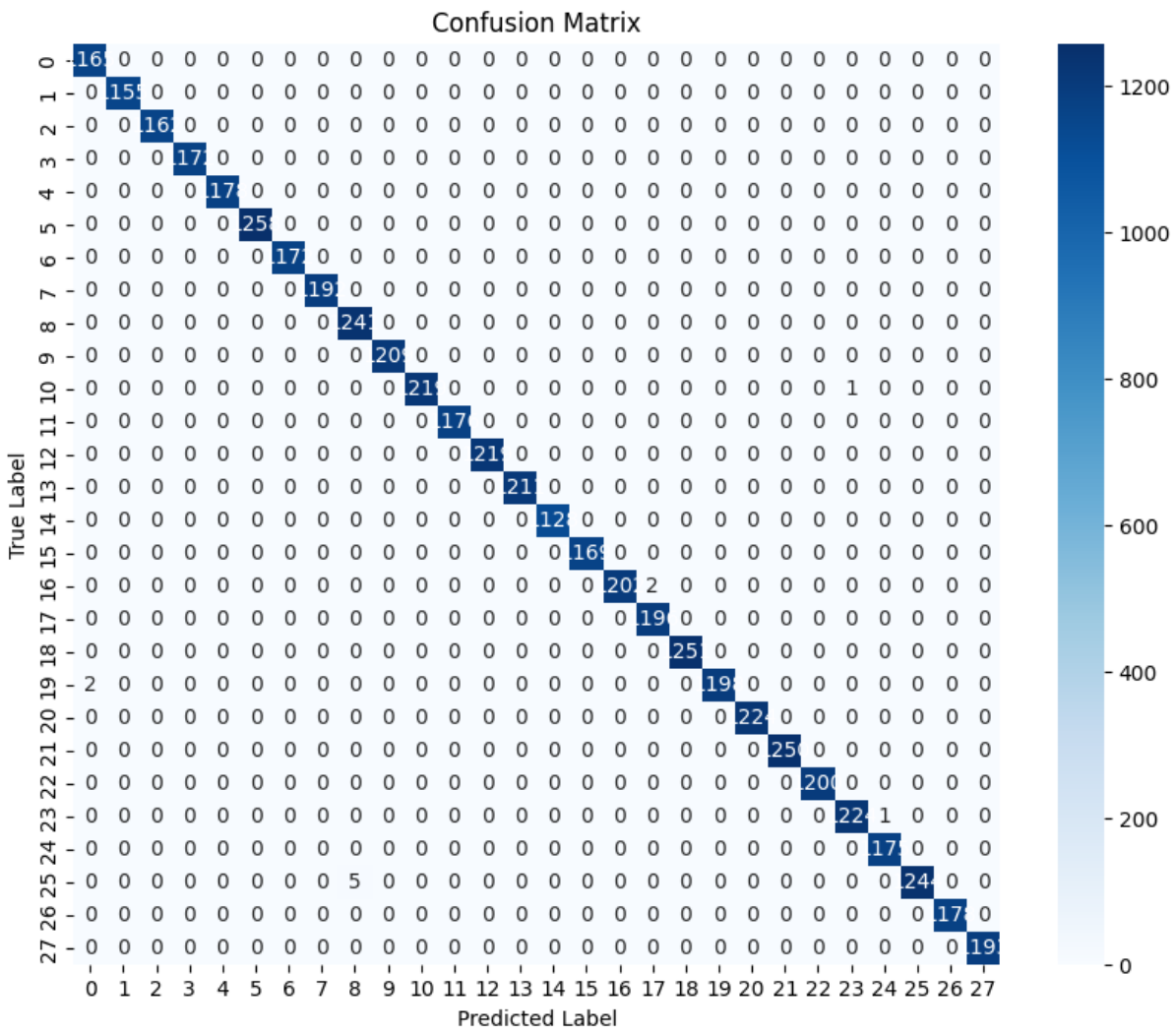

Figure 4.1: Confusion matrix for MobileNetV2 on the Kaggle dataset.

environment, its real-world applicability was initially uncertain.

The high accuracy obtained using this dataset indicated strong generalization over the diverse signs included in Kaggle's collection. However, this did not fully translate to real-time effectiveness, suggesting potential overfitting to the dataset's specific characteristics or a lack of representation of real-world variability.

Performance on the Custom Dataset

The custom dataset was specifically designed to replicate the conditions under which the sign language recognition system would operate, incorporating a wider range of variables such as lighting differences, diverse backgrounds, and variations in how signs are performed. Training MobileNetV2 on this custom dataset yielded substantial improvements, especially in real-time applications. This adaptation enabled the model to handle the nuances and variability of real-world use more effectively than the training performed with the Kaggle dataset.

The enhanced performance is not just about the high accuracy observed; the critical

improvement was the model's ability to correctly interpret signs under real-time conditions, making it more practically useful. Detailed performance metrics from the tests on the custom dataset include an impressive accuracy of 99.98% and a minimal loss of 0.0050. These figures underscore the model's robustness and its enhanced capability to function effectively in real-world scenarios.

Benefits of Training MobileNetV2 with the Custom Dataset

- *Reduced overfitting:* The custom dataset helped mitigate the overfitting observed with the Kaggle dataset by incorporating a more representative mix of real-world conditions.
- *Improved adaptability:* MobileNetV2 trained on the custom dataset showed greater adaptability and robustness, crucial for deployment in diverse environments where lighting, background, and other factors can significantly vary.
- *Focused training:* by tailoring the dataset to include specific challenges and scenarios, the training process became more focused, allowing MobileNetV2 to excel in the areas most needed for successful deployment.

This comparative analysis highlights the importance of dataset selection in training AI models. While the Kaggle dataset provided a strong foundation for initial model training and selection, the custom dataset was instrumental in refining MobileNetV2's capabilities to meet the practical demands of real-world sign language recognition.

4.5 Performance of Random Forest Classifier

The Random Forest Classifier was chosen as an alternative to deep learning models to assess its efficacy in managing the intricacies of sign language recognition. This model is well-regarded for its robustness and effectiveness in various classification scenarios. It was specifically trained and tested on a custom dataset that was captured using a web camera. This dataset was designed to include a broad range of real-world conditions, such as different lighting settings and backgrounds, which are crucial for enhancing the model's practical application in diverse environments.

4.5.1 Model Training

- *Feature Selection:* For the Random Forest Classifier, effective gesture recognition hinges on the precise selection and extraction of hand landmark features. These critical points on the hand capture the spatial configuration and orientation essential for distinguishing different signs.
- *Model Configuration:* The classifier is configured with 100 decision trees, ensuring comprehensive learning from the diverse gestures in the dataset. Each tree in the forest considers a random subset of these hand landmark features, which aids in better generalizing the model and reducing overfitting risks.

- *Training Details:* The training process involves multiple rounds where each decision tree learns from randomly selected data points through bootstrap sampling. Each tree uses a sample size equivalent to the original training set but selected with replacement, typically comprising about one-third of the total data. Additionally, feature bagging is employed at each split, considering a random subset of features (usually the square root of the total number of features) to further enhance the model's robustness against noise and variations in gesture presentation. This strategic approach ensures the classifier performs effectively across varied real-world scenarios.

4.5.2 Real-Time Evaluation

- *Accuracy Metrics:* In real-time testing, the Random Forest model demonstrated impressive performance on both the American and Arabic Sign Language custom datasets collected using a web camera. It achieved an accuracy of 99%, indicating not only the model's ability to generalize well beyond the training data but also its suitability for deployment in dynamic environments. This near-perfect accuracy underscores the model's effectiveness in a live setting, making it highly reliable for practical applications.
- *Response Time:* The model's prediction speed was optimized to support real-time interpretation without noticeable delays. This rapid response time is crucial for applications such as live communication or interactive sign language translation, where immediate feedback is essential for effective interaction.
- *Reliability:* The consistency in performance across various real-time testing scenarios with both ASL and ArSL datasets highlights the high reliability of the Random Forest Classifier. It accurately recognizes a wide range of sign language gestures, making it a valuable tool for enhancing communication accessibility for the mute community.

  This exploration into machine learning methods, particularly through the use of the Random Forest Classifier, underscores significant potential for combining traditional classification techniques with advanced feature extraction methods to create highly effective, efficient, and adaptable sign language recognition systems.

4.6 Hybrid Model: MobileNetV2 and Random Forest Classifier

In an effort to leverage the best of both worlds—the deep learning capabilities of MobileNetV2 and the robust classification power of the Random Forest—we developed a hybrid model. This approach was designed to utilize the high-level feature extraction capabilities of MobileNetV2 to capture intricate details from the sign language images, then pass these features to a Random Forest classifier for effective classification.

- *Feature Extraction:* MobileNetV2 was used to process input images and extract a dense representation of features. These features capture essential visual cues such as the shape, orientation, and positioning of hands, which are critical for accurate sign recognition.
- *Classification:* The extracted features were then fed into a Random Forest classifier. The decision to use Random Forest was based on its ability to handle high-dimensional data and provide reliable classifications, even with potential nonlinear relationships and interactions between features. The classifier was trained on a custom dataset captured with a web camera, covering both American and Arabic Sign Languages.

While innovative, the hybrid model faced several challenges:

- *Complexity in Integration:* Combining the outputs of a deep learning model with a machine learning classifier introduced complexities in data handling and integration. Ensuring that the feature sets were correctly interpreted by the Random Forest required careful alignment and tuning.
- *Limited Gesture Recognition:* Despite the theoretical advantages, the hybrid model initially recognized only 14 out of 29 gestures effectively. This limitation indicated possible issues in feature compatibility between the deep learning model and the machine learning classifier or insufficient training data for some of the less common gestures.
- *Performance Inconsistencies:* The model showed variations in performance across different sets of gestures, suggesting that while some gestures were captured effectively by the features extracted via MobileNetV2, others were not as well represented.

Despite these challenges and drawbacks, trying out the hybrid model provided valuable insights into what works and what does not when combining deep learning with traditional machine learning techniques. We concluded that selecting the right features is crucial, ensuring the models work well together is key, and thorough testing in various situations is necessary to ensure robust performance. This exploration highlighted the potential and the limitations of hybrid approaches in the field of sign language recognition.

4.7 Final Model Choice

After extensive testing and evaluation of various models, including CNNs and a hybrid approach, the decision was made to focus on the Random Forest Classifier for the final deployment of our sign language recognition system. This decision was primarily driven by the model's superior real-time performance and reliability, which are critical for the practical application of sign language recognition technology.

- *Real-time performance:* the Random Forest Classifier demonstrated exceptional accuracy and speed in real-time scenarios, outperforming other models

when deployed in environments similar to those in which it will be used post-deployment. Its ability to quickly and accurately classify gestures ensures that the system can operate effectively in real-time communication, which is vital for users relying on this technology for everyday interactions.

- *Reliability:* throughout various tests, the Random Forest model proved to be highly reliable, showing consistent performance across different datasets and conditions. This reliability, coupled with its robustness against overfitting and noise, makes it a dependable choice for ongoing use.

The journey through experimenting with CNNs and the hybrid model provided significant insights that were instrumental in shaping the final model choice.

- *Insights from CNNs:* while CNNs, particularly MobileNetV2, showed high accuracy in controlled test conditions, they fell short in real-time applicability when first deployed. The exploration of CNNs underscored the need for models not only to perform well on static datasets but also to adapt effectively to dynamic, real-world conditions where variability is much greater.
- *Learning from the hybrid model:* the hybrid model, combining MobileNetV2 and Random Forest, brought to light the challenges of integrating deep learning feature extraction with traditional machine learning classification. Although this approach did not yield the expected results, it highlighted the importance of feature compatibility and the complexities involved in model integration. The lessons learned from this experimentation were crucial in understanding the limitations and potential of such integrative approaches.

In conclusion, the exploration of various models through rigorous testing and evaluation led us to select the Random Forest Classifier. This model not only meets the technical non-functional requirements of accuracy and efficiency but also aligns well with the practical needs of end-users, ensuring that the sign language recognition system is both effective and user-friendly.

4.8 Conclusion

In this chapter, we took a deep dive into different models for sign language recognition. We looked at several advanced CNNs, like VGG16, MobileNetV2, MobileNetV3, DenseNet, and EfficientNet, and also checked out a machine learning method, the Random Forest Classifier. We even tried combining CNN features with the Random Forest's classification power. After all the tests we conducted, we chose the Random Forest Classifier as our final model because it performed exceptionally well in real-time and was very reliable across different scenarios. This decision came from thorough evaluations showing the model's efficiency and effectiveness in dynamic real-world settings.

In the next chapter, we will focus on how to practically apply our chosen model. In particular, we cover system design and implementation, making sure the technology performs well in real life. Our goal is to refine the system for better usability and

accessibility, making real-time sign language communication smoother and more intuitive for end-users.

Chapter 5

System Design and Implementation

5.1 Introduction

In this chapter, we transition from theoretical exploration to practical application within our sign language recognition project. We delve into the system design, detailing both what the system is intended to accomplish and how it ensures these goals are met with efficiency and reliability. Specifically, we outline the functional and non-functional requirements that guide the system's architecture, ensuring it is robust enough to handle real-world applications. Additionally, the chapter covers the hardware and software configurations that underpin our development and deployment strategies, providing a foundation for the detailed discussions on how the system is built and operates in diverse environments.

By the end of this chapter, you will have a clear understanding of the systematic approach behind our sign language recognition system, preparing you for a deeper look into its implementation and real-world functionality in subsequent sections.

5.2 System Requirements

5.2.1 Functional Requirements

Our sign language recognition system is built around several key functionalities that ensure it not only performs its primary task effectively but also enhances the user experience for individuals relying on it for communication.

- *Real-time gesture recognition:* at the heart of the system lies its capability to recognize and interpret sign language gestures in real-time. This feature ensures that there is minimal delay between the gesture being performed and its interpretation, which is crucial for seamless communication.

- *High accuracy in gesture interpretation:* the system is designed to achieve high accuracy rates in recognizing a wide range of gestures. This accuracy is vital for the system to be reliable and trustworthy, minimizing misunderstandings in communication.

- *User-friendly interface:* ease of use is paramount. The system features a user-friendly interface that allows users to interact with it straightforwardly, regardless of their technical expertise. This includes clear visual feedback and simple navigation cues to aid in its use during daily interactions.

5.2.2 Non-Functional Requirements

While the functional requirements focus on what the system does, the non-functional requirements address how well the system performs under various conditions.

- *Reliability*: is non-negotiable, as users depend on the system for accurate communication. The system is designed to operate consistently under different conditions, ensuring it remains operational and accurate when needed most.

- *Responsiveness:* the system is optimized for quick responses, ensuring that users experience no perceptible delays in communication. This is particularly important in real-time interactions where delays can hinder the flow of conversation.

- *Scalability:* as the adoption of the system grows, it is built to scale efficiently. Whether it is expanding the number of gestures recognized or supporting more simultaneous users, scalability ensures that the system's performance does not degrade as demand increases.

- *Maintainability:* with future advancements in technology and potential updates in sign language usage, the system is designed to be easily maintainable. This includes the ability to update software, add new functionalities, and adapt to changing hardware specifications without extensive modifications.

- *Environmental and hardware constraints*: given the deployment on diverse hardware platforms, from high-powered computers to more constrained devices like the Raspberry Pi, the system is designed to be adaptable. This flexibility ensures that performance does not suffer due to hardware limitations, and the system can deliver consistent functionality across different devices and environments.

5.3 System Architecture

5.3.1 High-Level Design

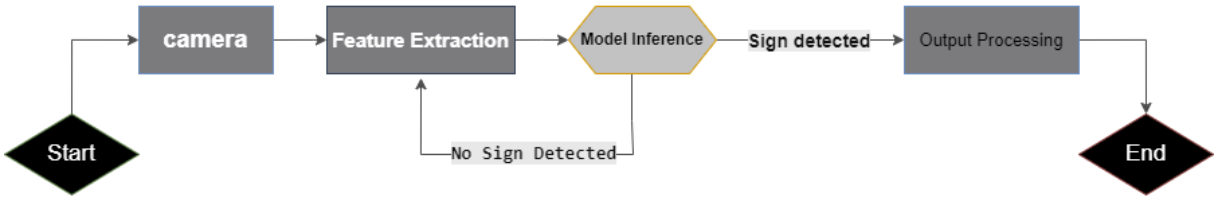

Figure 5.1: High-Level Design diagram

- *Input processing (camera)*: this component captures initial inputs, such as images or videos, using devices like webcams or specialized Raspberry Pi cameras. It optimizes capture settings to ensure the input is in a suitable format for processing. This optimization includes adjustments to resolution and lighting, enhancing the quality of the data for subsequent analysis.
- *Feature extraction:*
  MobileNetV2 on Raspberry Pi: In this stage, the system utilizes MobileNetV2, a deep learning model, primarily for its lightweight properties suitable for the limited processing power of the Raspberry Pi. This model preprocesses and extracts critical visual features from the input images, which involves resizing images and normalizing pixel values to prepare them for analysis. This is crucial for the Raspberry Pi deployment, focusing on leveraging MobileNetV2's efficiency despite its limited performance compared to more robust systems.
  Random Forest Classifier on Computer: Simultaneously, a different part of the system employs a Random Forest Classifier on a more powerful computer setup. This model is particularly effective at gesture recognition, using hand landmarks that identify specific points on the hands, such as fingertips and joints. These landmarks allow for detailed recognition of complex sign language gestures, enhancing the system's ability to interpret a wider range of signs accurately.
- *Model inference (decision point: sign detected or no sign detected):* as the core analytical engine of the system, this component uses the extracted features to interpret the gestures. Whether employing a CNN or a Random Forest model, it analyzes the features to determine the specific sign language gestures being made. This stage is critical in decision-making, determining whether a recognizable sign has been detected and processing it accordingly.

- *Output processing:* once gestures are identified, this final component formats the predictions into user-friendly outputs. It translates the recognized gestures into text, then spoken words, through text-to-speech technology using the `pyttx3` library, making the information accessible on a screen or through audio output. This allows users to receive immediate and clear communication based on the sign language interpreted.

5.3.2 Low-Level Design

Random Forest Classifier System on Computer

- *Data flow and processing:* this system captures real-time video input through a web camera. The captured frames are preprocessed to extract hand landmarks, which are then used as input for the Random Forest classifier.
- *Handling transitions and similar gestures:* the classifier sometimes experiences initial confusion between similar gestures, especially at the beginning of a sequence. However, it quickly adjusts to recognize the correct gestures as more data becomes available in real time.
- *Error correction and adaptation:* in cases where the system misinterprets gestures, especially those that are similar, it employs a short-term memory approach where it considers not just the current frame but a sequence to correct the prediction.
- *Feedback mechanism:* the system logs instances of misrecognition, which are later analyzed to improve model training and enhance gesture differentiation.

MobileNetV2 on Raspberry Pi

- *Data flow and processing*: due to the hardware constraints of the Raspberry Pi, the MobileNetV2 model, known for its efficiency in handling image data, processes input from the camera but at a significantly slower pace.
- *Interface integration:* a user interface displays the interpreted gestures. It includes functionality where users can manually correct the output by removing misidentified gestures, enhancing user interaction and accuracy.
- *Performance enhancement:* various strategies are being tested to reduce the latency and increase the processing speed of the system on the Raspberry Pi, such as model simplification and hardware acceleration techniques.
- *Error logging and feedback:* errors, especially those related to slow processing and misinterpretations, are logged. These logs help in pinpointing bottlenecks and areas for optimization to improve system responsiveness and accuracy.

5.4 Hardware and Software Configuration

5.4.1 Training Hardware

- *For American sign language:* all initial model training, including five different CNNs, was conducted on a laptop equipped with an Intel i7 8th generation processor, 16 GB of RAM, 512 GB of SSD, and dual graphics cards (Table **??**). After evaluating these models, MobileNetV2 was selected due to its superior performance. On the same first computer, further training and comparisons were carried out for MobileNetV2, a Random Forest classifier, and a hybrid model using datasets specifically tailored for American Sign Language.
- *For Arabic sign language:* a similar setup, using a second laptop with an Intel i7 10th generation processor, 16 GB of RAM, and 512 GB of SSD, was used to train the MobileNetV2 model, a Random Forest classifier, and a hybrid model, but this time focusing on Arabic sign language datasets (Table **??**).

Software and hardware	**Configuration**
American sign language	
Operating system	Windows 11 Pro
System type	64-bit operating system, x64-based processor
CPU	Intel i7 8th generation
RAM	16 GB
Disk capacity	512 GB
Arabic sign language	
Operating system	Windows 11 Pro
System type	64-bit operating system, x64-based processor
CPU	Intel i7 10th generation
RAM	16 GB
Disk capacity	512 GB

5.4.2 Deployment Hardware

- *Deployment on Raspberry Pi:* the MobileNetV2 model, optimized for American Sign Language on the first laptop, was deployed on a Raspberry Pi 3 Model B+ to handle real-time gesture recognition tasks. This deployment was chosen to assess the model's performance in a constrained hardware environment. The Raspberry Pi 3 Model B+ is equipped with a 32GB SD card.
- *Computer deployment:* both laptops, first for English and second for Arabic (Table **??**), were also used to deploy the systems for operational purposes, ensuring that each language's system was optimally tuned for its specific usage and dataset characteristics.

5.4.3 Software Configuration

- *Operating system:* both training laptops ran Windows 10, providing a stable and efficient platform for development and testing.
- *Raspberry Pi setup:*For the deployment of the MobileNetV2 model on the Raspberry Pi, significant adjustments were made to optimize performance within the device's limited resources. The Raspberry Pi OS was streamlined by disabling unnecessary processes to maximize available CPU and memory. TensorFlow settings were fine-tuned to accommodate the Pi's processing limits, involving adjustments for smaller batch sizes and optimizing for GPU acceleration when available. Additionally, thermal management was implemented to prevent CPU throttling and maintain consistent performance. These steps ensured the Raspberry Pi could efficiently handle real-time gesture recognition, demonstrating the potential of advanced machine learning models on budget-friendly hardware
- *Development environments and frameworks:* the projects heavily utilized Python for programming, given its extensive library support for data processing and machine learning. TensorFlow served as the core framework for developing and training the deep learning models due to its robust features and support for efficient model deployment.

5.4.4 software and harware setup

Table 5.1: Summary of Software and Hardware Configuration

Configuration	**American SL**	**Arabic SL**
Operating System	Win 10	Win 10
System Type	64-bit, x64	64-bit, x64
CPU	Intel i7-8th Gen	Intel i7-10th Gen
RAM	16 GB	16 GB
Storage	512 GB SSD	512 GB SSD
Graphics	Dual Cards	N/A
Add. Hardware	Raspberry Pi	Raspberry Pi
Dev. Env.	Python, TensorFlow	Python, TensorFlow
Pi Setup	Optimized TF	Streamlined OS, Optimized TF

5.5 Implementation Details

The implementation of our sign language recognition system commenced with setting up the necessary development environments on our two main laptops, one dedicated to American Sign Language and the other to Arabic Sign Language. Each laptop was equipped with all the required software, including Python and TensorFlow, to support the development and training of the models. This setup facilitated seamless integration of the programming environment with our training datasets.

5.5.1 Interface Development

A crucial part of the system is the user interface, designed to be intuitive and responsive. When a user performs a gesture in front of the webcam, the system first detects the hand using computer vision techniques. It then processes the gesture to recognize the corresponding sign language letter. This recognition process is initially set to a delay of five seconds to ensure accuracy but is under review for optimization to enhance user experience.

The interface dynamically assembles letters into words as gestures are recognized. A special gesture for *space* triggers the system to vocalize the constructed word using text-to-speech technology, effectively turning gestures into audible language. This functionality has been implemented on both laptops, tailored to handle both English and Arabic languages.

5.5.2 Deployment on Raspberry Pi

The deployment of the system on a Raspberry Pi was aimed at testing its efficiency in a more constrained hardware environment. While the MobileNetV2 model was optimized for deployment, challenges arose due to the limited processing power of the Raspberry Pi, which resulted in significant latency. The model's response time was slow, affecting real-time performance. In response to these challenges, efforts can be focused on system optimization to enhance processing speed without compromising accuracy. Adjustments include tweaking the model's architecture and exploring more efficient ways of handling data processing and model inference.

One of the primary challenges during implementation was ensuring that the system could perform efficiently in real-time on different hardware platforms. The difference in performance between the laptops and the Raspberry Pi highlighted the need for significant adjustments in the model's deployment strategy.

To overcome these, we have initiated a series of optimization trials aimed at reducing computational overhead and improving response times on the Raspberry Pi. This involves both software tweaks and potential hardware upgrades to better support the demands of real-time sign language translation.

5.6 Conclusion

This chapter has detailed the design and implementation process of our sign language recognition system, highlighting the functional and non-functional requirements to ensure both efficiency and reliability. Through discussions on system architecture at both high and low levels, we have illustrated how various components such as input processing, feature extraction, model inference, and output processing work cohesively to translate sign language into spoken words.

The hardware and software configurations emphasize the robust infrastructure supporting the system's operations. Utilizing high-performance computers for develop-

ment and training phases, alongside efficient deployment on Raspberry Pi, demonstrates our commitment to creating a versatile system adaptable across different technological environments. The implementation details provide insight into the practical steps taken to bring the system to life (from initial setup to final deployment) highlighting the challenges faced and the solutions crafted to ensure the system meets the rigorous requirements of real-time sign language interpretation.

In the next chapter, we will examine the performance of the implemented sign language recognition system.

Chapter 6

Results, Discussion, and Conclusion

6.1 Introduction

In this chapter, we examine how our sign language recognition system has performed. Our goal from the beginning was to develop a system that accurately interprets sign language in real-time, making it a practical tool for those who rely on sign language daily.

Over the previous chapters, we have outlined our journey (from the early stages of concept development to the detailed processes of building and refining our technology). In this chapter, we provide detailed results from our testing, showing how well the system has met our high standards for accuracy and usability. We also discuss insights from real-world usage, shedding light on how the system operates in the hands of end-users. This chapter aims to provide a comprehensive overview of the tangible outcomes of our efforts, supported by quantitative data and enriched with qualitative experiences.

6.2 Evaluation Metrics

Accurate assessment of the system's performance is critical to ensure it meets the needs of its users effectively. For our sign language recognition system, we focused on several key metrics that are particularly relevant to classification problems like ours.

Accurate assessment of the system's performance is critical to ensure it meets the needs of its users effectively. For our sign language recognition system, we focused on several key metrics that are particularly relevant to classification problems.

- *Accuracy*: this is the most straightforward metric, representing the overall percentage of correct predictions made by the system out of all predictions made. High accuracy is crucial for ensuring the system is reliable and can be trusted to perform well in real-world applications.

- *Precision*: measures the correctness achieved in positive predictions. That is, it reflects the proportion of positive identifications that were actually correct. This metric is vital when the cost of a false positive is high, ensuring that when a sign is recognized, it is indeed the correct interpretation.
- *Recall (Sensitivity)*: this metric measures the ability of the model to detect all relevant instances correctly. In the context of sign language recognition, a high recall rate means the system is capable of recognizing almost all gestures presented to it, which is crucial for effective communication.
- *F1 Score*: is the harmonic mean of precision and recall. It is a better measure than accuracy in scenarios where the class distribution is imbalanced (which is often the case in real-world scenarios). This metric is particularly useful for our system as it balances the precision and recall, providing a single score to gauge the model's performance.

6.3 Comprehensive Performance Evaluation

6.3.1 Model Accuracy and Loss Metrics

- **Performance on Computers: Random Forest Classifier**
  For the final implementation on computers, we decided to utilize the Random forest classifier based on its superior performance during testing phases. The classifier demonstrated an impressive accuracy of 99% across both American and Arabic sign language datasets, with a minimal loss of 0.01. This high level of accuracy and the low loss indicate that the Random Forest Classifier was highly effective at recognizing and interpreting sign language gestures accurately, making it our model of choice for deployment on standard computing hardware.
- **Deployment on Raspberry Pi: MobileNetV2**
  On the Raspberry Pi, we deployed the MobileNetV2 model due to its lightweight architecture, which is suitable for the limited computational resources of the Raspberry Pi. The MobileNetV2 model achieved a notable accuracy of 99% with a loss of 0.02. Despite these promising metrics, we encountered significant challenges with the system's performance speed on the Raspberry Pi. The model operated much slower than anticipated, which limited our ability to assess its real-time functionality effectively. This slowdown underscores the need for further optimization to enhance the model's efficiency on low-power devices.

6.3.2 Confusion Matrices and Graphical Analysis

- **Random Forest Classifier Performance on English Custom Dataset**

  The confusion matrix for the Random Forest Classifier trained on the English custom dataset demonstrates robust accuracy. Most gestures were cor-

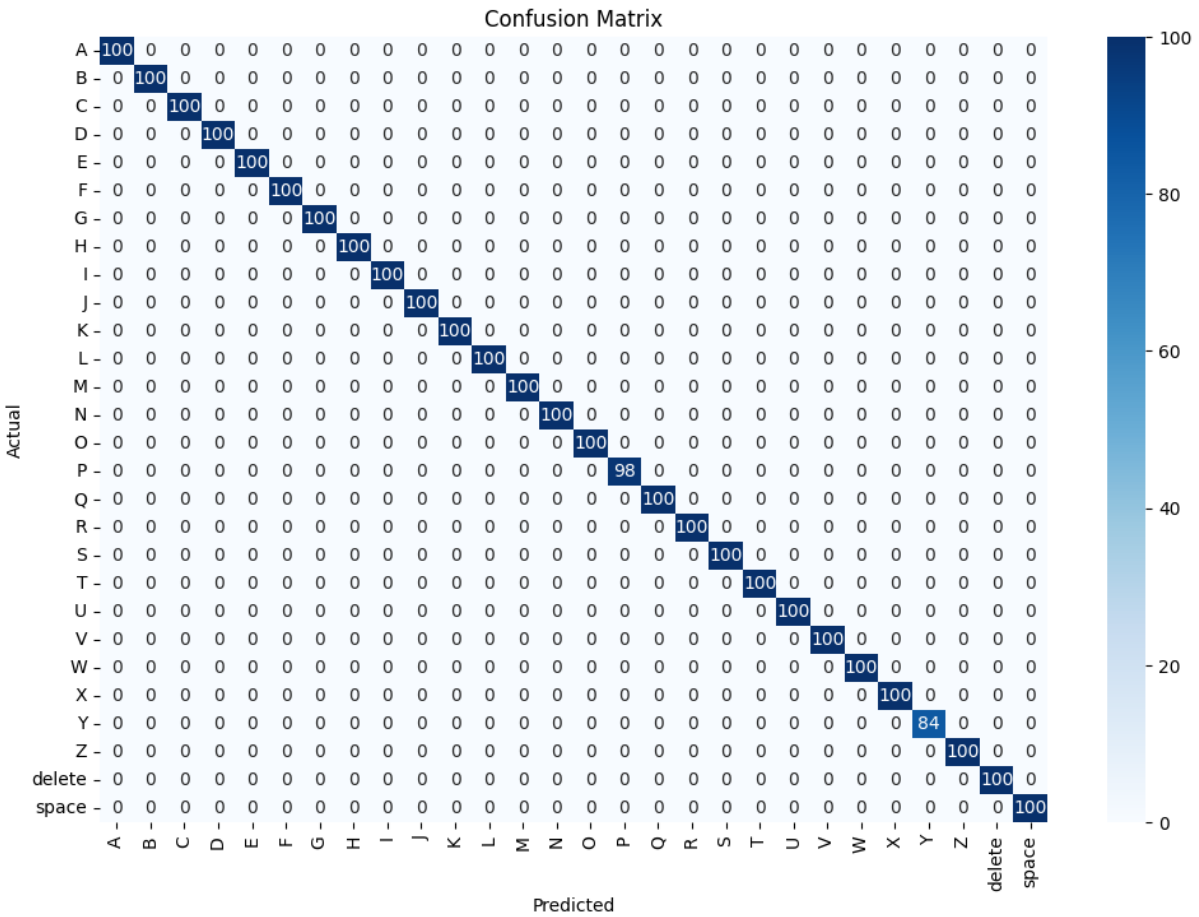

Figure 6.1: Confusion Matrix for the Random Forest Classifier on the English custom dataset

rectly identified with near-perfect scores, indicating the classifier's effectiveness. Minor misclassifications for gestures like 'P' and 'Y' suggest areas for slight improvement, but overall, the high accuracy across the board confirms the model's reliability in sign language recognition.

- **Random Forest Classifier Performance on Arabic Custom Dataset**

- **MobileNetV2 Performance on Custom Dataset**

6.4 Real-Time System Performance

6.4.1 Interface Functionality and Real-Time Testing

system employs a highly interactive interface coupled with real-time processing capabilities to deliver immediate feedback based on sign language gestures. Here's a detailed overview of how our system handles input and outputs in real-time:

- **Input Processing:**
  The system continuously captures video frames using a standard webcam, processing each frame in real time to capture dynamic hand movements as the user signs. Employing MediaPipe Hands, a robust hand tracking solution, the system identifies and tracks hand landmarks such as fingertips and joints within each video frame. This tracking is essential for accurately interpreting sign gestures.
- **Gesture Recognition Process:**
  Once hand landmarks are detected, the system extracts relevant features from their spatial positioning, which are critical for the subsequent gesture recognition phase. Designed to handle ambiguities or partial gestures, the system continuously updates its recognition based on the latest inputs. If a gesture is not clearly recognized initially, additional input frames help the system correct and finalize its interpretation.
- **Output Processing:**
  After a gesture is recognized, the corresponding letter or command (such as 'space' or 'delete') is converted into text displayed on the interface. If the 'space' gesture is detected, the system uses text-to-speech technology to vocalize the constructed word, providing an audible output of the signed input. The user interface updates in real-time to show the recognized letters and words as they are signed, offering immediate visual feedback for the user to verify that their gestures are being correctly interpreted by the system.
- **Error Handling and System Adjustments:**
  If an incorrect letter is detected or there's a delay in recognition, the system allows for quick corrections. The 'delete' gesture enables users to remove the last character entered, ensuring that errors can be promptly fixed without disrupting the flow of communication. Despite the system's high accuracy and responsiveness on computers, the Raspberry Pi deployment has encountered performance issues due to its slower processing capabilities. We are actively working to optimize the software to reduce lag and ensure smoother operation on these lower-power devices.

6.4.2 Interface Overview for Hand Gesture Recognition System

feigure..... The image displays the user interface of our Hand Gesture Recognition system during a live session. This interface is crucial for providing real-time feedback to users as they perform sign language gestures.

Key Features of the Interface

- **Live Video Feed:** The main portion of the interface shows a live video feed capturing the user's hand gestures. This visual feedback is essential for users to adjust their gestures if necessary and confirm that their signs are being captured accurately by the camera.

- **Text Display Area:** On the right side of the interface, the recognized gestures are translated into text. In this example, the letters "ABC" are displayed, indicating that the system has successfully recognized and converted the gestures into corresponding text. This area updates dynamically as the user continues to sign.
- **Real-Time Interaction:** The interface supports real-time interaction, updating the display as new gestures are recognized. This feature is vital for continuous communication, allowing users to see immediate translations of their sign language.
- **User Feedback for Accuracy:** The interface also serves as a feedback mechanism, where users can visually confirm that the system is interpreting their gestures correctly. If a mistake is detected, users can adjust their signing or utilize correction gestures, such as the "delete" gesture, to remove the last recognized character.

Functionality and Usage

- The system uses advanced computer vision and machine learning algorithms to detect hand landmarks and interpret gestures. Each frame from the video feed is processed to identify hand positions and movements, which are then matched against a pre-trained model to recognize specific gestures.
- Upon recognizing a gesture, the system updates the text display area almost instantly, making it an effective tool for real-time sign language communication.
- The system is designed to be user-friendly, requiring minimal setup and providing a straightforward interface that can be used comfortably in various settings, including home, work, and educational environments.

6.5 Future Work

As we look towards the future of our sign language recognition system, several enhancements and research directions stand out as promising avenues for development. These improvements aim to refine the system's current capabilities while also extending its utility and accessibility.

6.5.1 Potential Enhancements

- **Expanding Gesture Recognition Capabilities:** One of the key areas for enhancement is the integration of additional gesture recognition features. This includes recognizing more complex sign language constructs such as idiomatic expressions and non-manual signals, which involve facial expressions and body movements.

- **Including More Sign Languages:** Currently, the system supports American and Arabic Sign Language. Expanding the system to include other sign languages, such as British Sign Language (BSL) and Indian Sign Language (ISL), would make it more inclusive and beneficial to a broader user base. This expansion requires collecting and training on diverse datasets to accurately capture the nuances of each language.
- **Improving Hardware Efficiency:** Enhancing the system's hardware efficiency is crucial for improving portability and usability, especially on devices with limited processing power like the Raspberry Pi. Efforts to optimize algorithms and perhaps incorporate edge computing could lead to significant gains in speed and responsiveness, making the system more practical for everyday use.

6.5.2 Future Research Directions

- **Real-World Deployment Studies:** Conducting extensive real-world deployment studies could provide invaluable insights into the system's performance in various environments. This research would involve deploying the system in settings such as educational institutions, public services, and homes to gather data on user interaction, system reliability, and environmental adaptability.
- **User-Centered Design Improvements:** Future research could also focus on a user-centered design approach, where feedback from a diverse range of users is used to tailor the system more closely to user needs. This approach would likely uncover additional functionalities that could enhance user interaction and satisfaction.
- **Cross-Device Compatibility Studies:** Exploring the system's compatibility across different devices and platforms would ensure that it can be used seamlessly by a wider audience. Studies could focus on optimizing the system for various operating systems and hardware configurations, ensuring consistent performance regardless of the user's device.

6.6 Conclusion

This project is a big step forward in making technology that helps people who use sign language to communicate better. By improving this sign language recognition system, we show how technology can help break down barriers that stop people from fully joining in on everyday activities. This work not only helps people who are mute or hard of hearing but also opens up possibilities for using similar technology to help others, like those with physical or cognitive challenges.

The success of this system proves how important it is to keep making new technologies that make communication easier for everyone. As technology gets better, it brings new chances to make tools that are more helpful and easy to use. We're seeing that artificial intelligence and other advanced technologies could make these tools even smarter, adapting to what users need right away.

Looking back at this project, it's a guide for making new tech in the future and a reminder of the big impact thoughtful technology can have on making life better for everyone. Moving forward, we all need to keep working on new ways to make sure technology includes everyone, making sure no one is left out as things keep changing.

Chapter 7

Conclusions and Perspectives

Reflecting on the entirety of this thesis and the comprehensive journey undertaken in the development of the sign language recognition system, several key themes and achievements stand out. This work tackled technical challenges and contributed to a field that sits at the intersection of technology, accessibility, and human interaction. The core of this project was the development of a robust sign language recognition system that leverages advanced machine learning techniques and innovative hardware solutions. By employing models like MobileNetV2 and the Random Forest classifier, the system demonstrated high accuracy and efficiency in recognizing sign language gestures. This technical achievement underscores the potential of integrating cutting-edge technology into practical, user-oriented applications.

One of the primary goals of this project was to enhance communication accessibility for individuals who use sign language. The system developed provides a real-time, intuitive means for users to translate sign language into text and speech, bridging communication gaps and fostering inclusivity. Such advancements are crucial for enhancing the quality of life and independence of individuals with hearing impairments, enabling them to communicate more freely and effectively in a variety of settings. Throughout the development process, various challenges were encountered, including optimizing performance for low-resource hardware like the Raspberry Pi and ensuring the system's usability across different languages and environments. These challenges were not merely obstacles but opportunities for learning and improvement. Each problem addressed and each solution implemented contributed to a deeper understanding of both the technological and user-centric aspects of system design.

Looking forward, there is a clear path for further research and development. The potential for expanding the system to include additional sign languages, enhancing real-time processing capabilities, and integrating more adaptive learning features presents exciting opportunities for future work. Moreover, the deployment of this technology in real-world settings and its adaptation based on user feedback will be crucial for its evolution and relevance. Ultimately, this thesis contributes to the broader field of assistive technologies by demonstrating how thoughtful application of machine learning and careful system design can lead to significant advancements in accessibility. It also highlights the ongoing need for technological innovation that

is inclusive and responsive to the needs of all users, particularly those from disabled communities.

In sum, this thesis showcases the successful development of a sign language recognition system and sets a foundation for future innovations in assistive technology. It is a testament to the impact that technology can have on improving human lives, especially when it is directed towards breaking down barriers and enhancing communication for those who need it most.

Bibliography

[1] Muhammad Albrham. *RGB Arabic Alphabets Sign Language Dataset*. Accessed: 2024-06-07. 2021. URL: https://www.kaggle.com/datasets/muhammadalbrham/rgb-arabic-alphabets-sign-language-dataset.

[2] Charlotte Lee Baker-Shenk and Dennis Cokely. *American Sign Language: A Teacher's Resource Text on Grammar and Culture*. Gallaudet University Press, 1980. ISBN: 093032384X.

[3] Casey Bennett and Thomas Doub. "Temporal Modeling in Clinical Artificial Intelligence, Decision-Making, and Cognitive Computing: Empirical Exploration of Practical Challenges." In: *SIAM Workshop on Data Mining for Medicine and Healthcare (DMMH)* 1 (Apr. 2014), pp. 1–10.

[4] Jason Brownlee. *Overcoming the Curse of Imbalanced Class Data in Machine Learning*. Machine Learning Mastery. 2016. URL: https://machinelearningmastery.com/tactics-to-combat-imbalanced-classes-in-your-machine-learning-dataset/.

[5] Timothy M. Bullock, Nikhil Sharma, and Iain Matthews. "Sign language machine translation using conditional random fields." In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 39.7 (2017), pp. 1424–1437.

[6] Nitesh V. Chawla, Nathalie Japkowicz, and Aleksander Kotz. *Editorial: Special issue on learning from imbalanced data sets*. 2004.

[7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[8] G. Guarino et al. "Temporal information integration for video semantic segmentation." In: May 2020, pp. 8545–8551. DOI: 10.1109/ICRA40945.2020.9197204.

[9] Huan He and Enrique A. Garcia. "Learning from imbalanced data." In: *IEEE Transactions on Knowledge and Data Engineering* 21.9 (2009), pp. 1263–1284.

[10] Andrew Howard et al. *Searching for efficient mobilenetv3*. 2019. eprint: arXivpreprintarXiv 1905.02244.

[11] Gao Huang et al. *Densely Connected Convolutional Networks*. 2018. arXiv: 1608.06993 [cs.CV].

[12] Gareth James et al. *An Introduction to Statistical Learning*. Vol. 112. Springer Texts in Statistics. Springer, 2013.

[13] Kaggle. *Kaggle*. Accessed: 2024-06-07. 2024. URL: https://www.kaggle.com.

[14] Yannic LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep learning." In: *Nature* 521.7553 (2015), pp. 436–444.

[15] John Locke. "A Theory of Neurolinguistic Development." In: *Brain and language* 58 (July 1997), pp. 265–326. DOI: 10.1006/brln.1997.1791.

[16] Kapil Londhe. *American Sign Language*. Accessed: 2024-06-07. 2020. URL: https://www.kaggle.com/datasets/kapillondhe/american-sign-language.

[17] Tom M Mitchell. *Machine Learning*. McGraw-Hill, 1997.

[18] Mark Sandler et al. *MobileNetV2: Improved mobile network architecture with inverted residual blocks*. 2018. eprint: arXivpreprintarXiv:1801.04381.

[19] Arushi Seth and Shanu Sharma. "Semantic Segmentation: A Systematic Analysis From State-of-the-Art Techniques to Advance Deep Networks." In: *Journal of Information Technology Research* 15 (Jan. 2022), pp. 1–28. DOI: 10.4018/JITR.299388.

[20] Ahmad Al-Shamayleh et al. "AUTOMATIC ARABIC SIGN LANGUAGE RECOGNITION: A REVIEW, TAXONOMY, OPEN CHALLENGES, RESEARCH ROADMAP AND FUTURE DIRECTIONS." In: *Malaysian Journal of Computer Science* 33 (Oct. 2020), pp. 306–343. DOI: 10.22452/mjcs.vol33no4.5.

[21] Connor Shorten and Taghi M. Khoshgoftaar. "A survey on image data augmentation for deep learning." In: *Journal of Big Data* 6.1 (2019). DOI: 10.1186/s40537-019-0197-0.

[22] Karen Simonyan and Andrew Zisserman. *Very deep convolutional networks for large-scale image recognition*. 2014. eprint: arXivpreprintarXiv:1409.1556.

[23] Mingxing Tan and Quoc V. Le. "EfficientNet: Rethinking model scaling for convolutional neural networks." In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. 2019, pp. 6105–6114.

[24] C. Valli and C. Lucas. *Linguistics of American Sign Language: An Introduction*. Clerc books. Gallaudet University Press, 2000. ISBN: 9781563680977.

[25] Wikipedia contributors. *Random forest — Wikipedia, The Free Encyclopedia*. [Online; accessed 8-June-2024]. 2024. URL: https://en.wikipedia.org/w/index.php?title=Random_forest&oldid=1226321951.